Lecture 16: Hypertext Transfer Protocol (HTTP) II

Objectives:

- Learn about classes in C# that simplify the creation of HTTP Clients
- Learn how to pass a HTTP request through a proxy server
- Learn how to write a simple Http Server

1. The WebClient Class

We saw in lecture 15, how to use the TcpClient class to create a HTTP client.

To create a HTTP server, we use the TcpListener or Socket class.

In addition, C# provides a number of classes in the System.Net namespace that simplify the creation of HTTP clients. The simplest of these classes is the **WebClient** class.

The following are some of the properties and methods of the WebClient class.

Properties

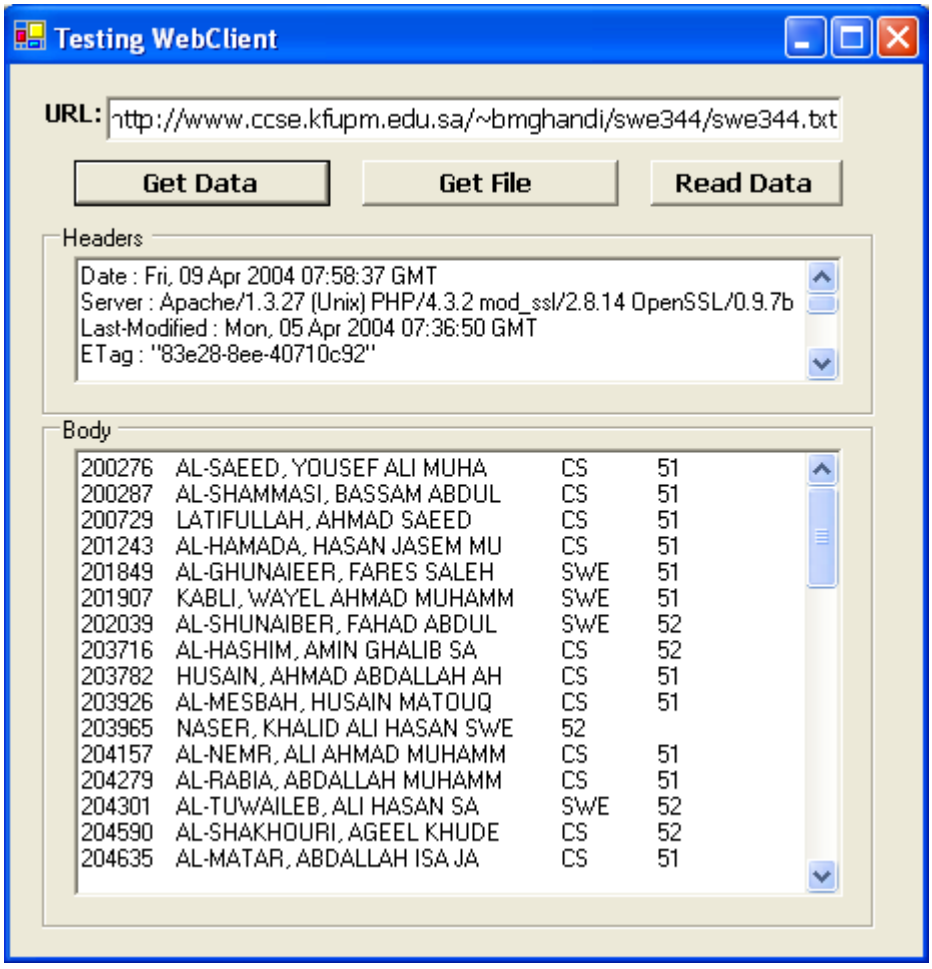
Credentials	Gets or sets the network credentials used to authenticate the request with the Internet resource.
Headers	Gets or sets a collection of header name/value pairs associated with the request.
ResponseHeaders	Gets a collection of header name/value pairs associated with the response.

Methods

byte[] DownloadData(string uri);	Downloads data from a resource with the specified URI.
void DownloadFile(string uri, string fileName);	Downloads data from a resource with the specified URI to a local file.
Stream OpenRead(string uri);	Opens a readable stream for the data downloaded from a resource with the specified URI.
Stream OpenWrite(string uri); Stream OpenWrite(string uri, string	Opens a stream for writing data to a resource with the specified

method);	URI.
byte[] UploadData(string uri, byte[] data); byte[] UploadData(string uri, string method, byte[] data);	Uploads a data buffer to a resource with the specified URI.
byte[] UploadFile(string uri, string fileName); byte[] UploadFile(string uri, string method, string fileName);	Uploads a local file to a resource with the specified URI.

The following example shows how to use the WebClient class:




```
private WebClient client;
public WebClientExample()
{
      InitializeComponent();
      client = new WebClient();
}

void OnGetData(object sender, System.EventArgs e)
{
      try {
```

```
            byte[] data = client.DownloadData(urlBox.Text);
            resultBox.Text = Encoding.ASCII.GetString(data);
            ShowHeaders();
      }
      catch (WebException ex) {
            MessageBox.Show(ex.Message, "Exception");
      }
}

void OnGetFile(object sender, System.EventArgs e)
{
      try {
            string fname =
urlBox.Text.Substring(urlBox.Text.LastIndexOf("/")+1);
            client.DownloadFile(urlBox.Text, fname);
            resultBox.Text = "File Downloaded";
            ShowHeaders();
      }
      catch (WebException ex) {
            MessageBox.Show(ex.Message, "Exception");
      }
}

void OnReadData(object sender, System.EventArgs e)
{
      try {
            StreamReader reader = new
StreamReader(client.OpenRead(urlBox.Text));
            String line;
            resultBox.Text = "";
            while ((line=reader.ReadLine())!= null)
                  resultBox.Text += line + "\r\n";
            reader.Close();
            ShowHeaders();
      }
      catch (WebException ex) {
            MessageBox.Show(ex.Message, "Exception");
      }
}

void ShowHeaders() {
      WebHeaderCollection whc = client.ResponseHeaders;
      headerBox.Text = "";
      for(int i=0; i<whc.Count; i++) {
            headerBox.Text += whc.GetKey(i) + " : " + whc.Get(i) + "\r\n";
      }
 }
```

2. The NetworkCredential class

Some web-servers do not allow anonymous access for some resources. For such servers, an instance of the **NetworkCredential** class is created and assigned to the **Credentials** property of the **WebClient** instance before sending the request.

The NetworkCredential class has the following constructors:

```
NetworkCredential();
NetworkCredential(string userName, string password);
NetworkCredential(string userName, string password, string domain);
```

It also has properties: Domain, Password and UserName.

3. The WebRequest, WebResponse and related classes.

For more advanced HTTP client applications, the WebRequest and WebResponse classes should be used.

These classes provide more methods and properties for manipulating the headers, including a Proxy property, which is used to specify a proxy server.

However, the WebRequest and WebResponse are abstract classes. To create objects, their subclasses are used depending on the type of the URI.

The subclasses of WebRequest are HttpWebRequest and FileWebRequest.

Similarly, WebResponse has HttpWebResponse and FileWebResponse as concrete subclasses.

The following are some of the properties and methods of the HttpWebRequest class:

Properties

Accept	Gets or sets the value of the **Accept** HTTP header.
Address	Gets the URI of the Internet resource that actually

	responds to the request.
AllowAutoRedirect	Gets or sets a value that indicates whether the request should follow redirection responses.
ClientCertificates	Gets the collection of security certificates associated with this request.
Connection	Gets or sets the value of the **Connection** HTTP header.
ContentLength	Gets or sets the **Content-length** HTTP header.
ContentType	Overridden. Gets or sets the value of the **Content-type** HTTP header.
Credentials	Overridden. Provides authentication information for the request.
HaveResponse	Gets a value indicating whether a response has been received from an Internet resource.
Headers	Gets a collection of the name/value pairs that make up the HTTP headers.
IfModifiedSince	Gets or sets the value of the **If-Modified-Since** HTTP header.
KeepAlive	Gets or sets a value indicating whether to make a persistent connection to the Internet resource.
MediaType	Gets or sets the media type of the request.
Method	Gets or sets the method for the request.
Pipelined	Gets or sets a value indicating whether to pipeline the request to the Internet resource.
ProtocolVersion	Gets or sets the version of HTTP to use for the request.
Proxy	Gets or sets proxy information for the request.
SendChunked	Gets or sets a value indicating whether to send data in segments to the Internet resource.
UserAgent	Gets or sets the value of the **User-agent** HTTP header.

Methods

Abort	Cancels a request to an Internet resource.
BeginGetRequestStream	Begins an asynchronous request for a Stream instance to use to write data.
BeginGetResponse	Begins an asynchronous request to an Internet resource.

EndGetRequestStream	Ends an asynchronous request for a Stream instance to use to write data.
EndGetResponse	Ends an asynchronous request to an Internet resource.
GetRequestStream	Gets a Stream instance to use to write request data.
GetResponse	Returns a response from an Internet resource.

4. WebProxy and GlobalProxySelection classes

To communicate through a proxy, an instance of the WebProxy class must be created.

The proxy instance is assigned to the **Proxy** property of the WebRequest instance or assigned globally using the GlobalProxySelection class.

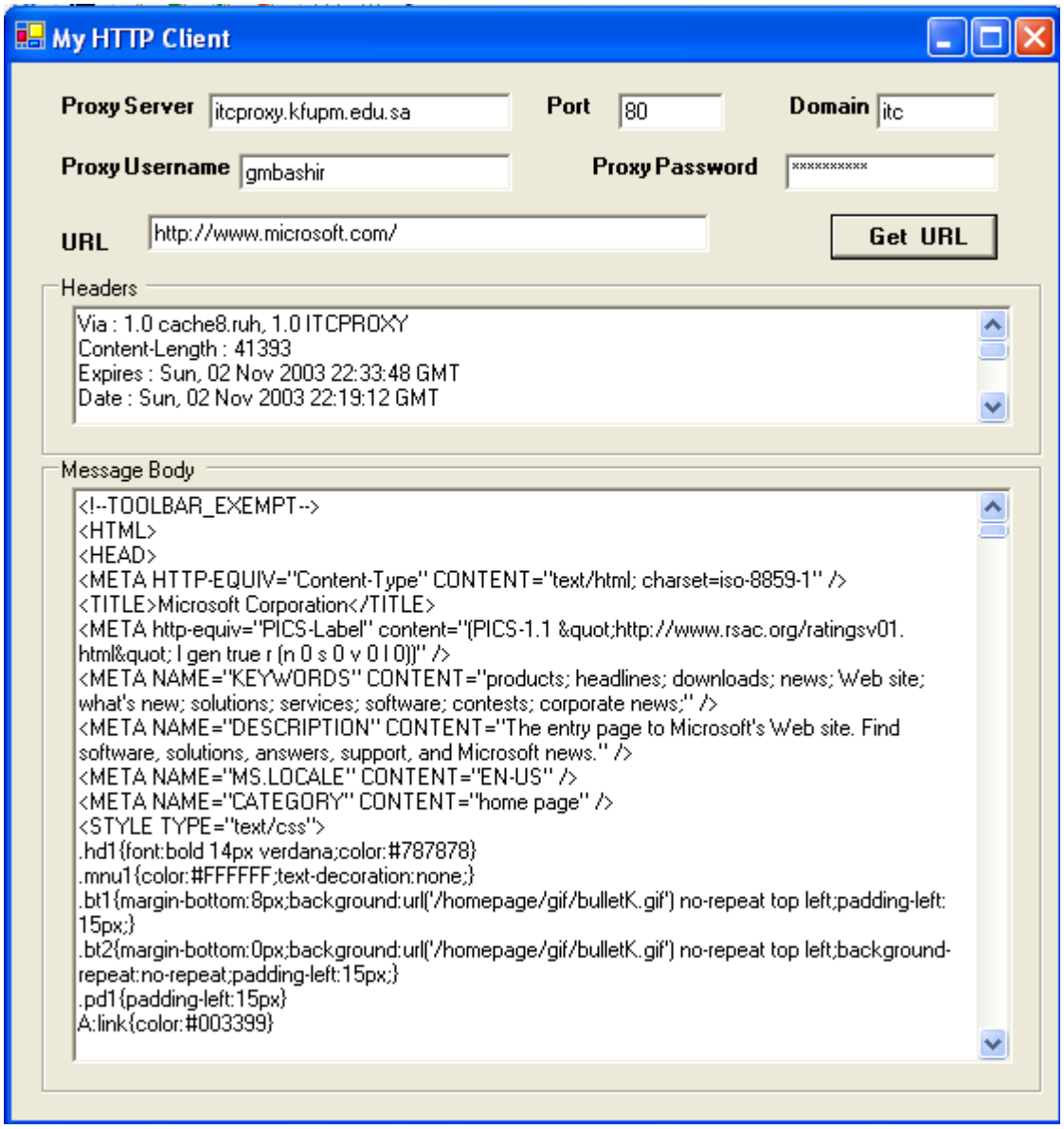
The most complete version of the WebProxy constructor has the following form:

public WebProxy(string Address, bool BypassOnLocal, string[] BypassList, ICredentials Credentials);

The **GlobalProxySelection** class is used to define a default proxy which all instances of Web Request will use. It has a property, **Select**, which is assigned the WebProxy instance.

Notice that GlobalProxySelection class can also be used with the **WebClient** class.

Example:
The following example shows a HTTP client, which can be used to communicate with servers even beyond the local network.




```
void OnGetUrlClicked(object sender, System.EventArgs e)
{
      headerBox.Text = "";
      resultBox.Text = "";
      NetworkCredential myCred = null;
      WebProxy proxyObject = null;

      if ( proxyPasswordBox.Text != "")
      {
            myCred = new NetworkCredential(proxyUsernameBox.Text,

proxyPasswordBox.Text, proxyDomainBox.Text);
            proxyObject= new
WebProxy(proxyServerBox.Text+":"+proxyPortBox.Text,
                                                      true,null,
myCred);
            //GlobalProxySelection.Select = proxyObject;
      }
```

```
      if (!hasProtocol(urlBox.Text))
            urlBox.Text = "http://"+ urlBox.Text;

      //if no file name is given append slash
      if (urlBox.Text.LastIndexOf('/') == urlBox.Text.LastIndexOf("//")+1)
            urlBox.Text +="/";

      WebHeaderCollection whc;
      WebResponse result = null;
      try {
            WebRequest req = WebRequest.Create(urlBox.Text);
            if (proxyObject != null)
                  req.Proxy = proxyObject;
         result = req.GetResponse();
         Stream rs = result.GetResponseStream();
         StreamReader sr = new StreamReader(rs);
         resultBox.Text=sr.ReadToEnd();

            whc = result.Headers;
            for (int i=0; i<whc.Count; i++) {
                  headerBox.Text += whc.GetKey(i) + " : " + whc.Get(i) +
"\r\n";
            }
            result.Close();
      }
      catch(Exception) {
            resultBox.Text = "Exception has occured";
      }
}

private bool hasProtocol(string s) {
      return s.IndexOf("//") != -1;
}
```

5. Creating a Http Server

Although there are a number of classes specifically designed to help create Http clients, there are no such helper classes for creating Http Servers.

Like any TCP server, a http server is created using the **TcpListener** class or using the **Socket** class.

The following example shows a simple Http 1.0 server that supports only the GET method.

```
using System;
using System.Collections;
```

```
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Threading;

class SimpleHttpServer {
     static int count = 0;

     public static void Main(string[] args)
     {
          string docFolder = Directory.GetCurrentDirectory();
          int portNumber = 8080;
          TcpListener tcpListener = new TcpListener(IPAddress.Any,
portNumber);
          tcpListener.Start();

          Console.WriteLine("Waiting for a connection on port 8080 ... ");

          while (true)
          {
             TcpClient client = tcpListener.AcceptTcpClient();
               Console.WriteLine("Visitor Count: "+ (++count));
               ConnectionHandler handler = new ConnectionHandler(client);
               ThreadPool.QueueUserWorkItem(new
WaitCallback(handler.HandleConnection), docFolder);
          }
     }
}

class ConnectionHandler
{
     TcpClient client;
     NetworkStream stream;
     StreamReader reader;
     StreamWriter writer;
     string docFolder;
     string request;

     public ConnectionHandler(TcpClient client) {
          this.client = client;
          stream = client.GetStream();
          reader = new StreamReader(stream);
          writer = new StreamWriter(stream);
     }

     public void HandleConnection(Object state) {
          docFolder = (string) state;

          request = reader.ReadLine(); //read the request;

          string input;
          //read rest of the request
          while ((input = reader.ReadLine()).Length != 0);
```

```
          string[] token = request.Split(' ');
          if (token.Length == 3 && token[0] == "GET" &&
              token[1].StartsWith("/") && token[2].StartsWith("HTTP")) {

               string fileName = token[1].Replace('/', '\\');
               fileName = docFolder + fileName;
               if (File.Exists(fileName)) {
                    writer.Write("HTTP/1.0 200 OK\r\n"); //send response
line
                    SendHeaders(fileName);
                    SendFile(fileName);
               }
               else
                    WriteError(404, "Not Found");
          }
          else {
              WriteError(400, "Bad Request");
          }
          stream.Close();
          client.Close();
     }

     public void WriteError(int status, string message) {
          writer.Write("HTTP/1.0 " + status + " " + message + "\r\n");
          writer.Write("\r\n");
          writer.Flush();
     }

     public void SendHeaders(string fileName) {
          FileInfo file = new FileInfo(fileName);
          writer.Write("Content-Length: " + file.Length + "\r\n");
          writer.Write("\r\n"); //blank line here!
          writer.Flush();     //need to flush the headers
     }

     public void SendFile(string fileName) {
          FileStream file = new FileStream(fileName, FileMode.Open,
FileAccess.Read, FileShare.Read);

          byte[] data = new byte[4096];
          while (file.Position < file.Length) {
               int read = file.Read(data, 0, data.Length);
               stream.Write(data, 0, read);
          }
          stream.Flush();
          file.Close();
     }
}
```